

Anexo: Detalles de implementación

Noel Pérez Calvo

22 de mayo de 2026

Anexo: Detalles de implementación

El algoritmo descrito en el Capítulo ?? se materializa en un sistema compuesto por varios módulos independientes, cada uno responsable de una etapa del proceso. A continuación se describen estos módulos, se detallan los recursos que requieren y se resume el formato de los resultados que producen.

0.1. Descripción de los módulos

La implementación sigue el orden del pipeline presentado en el pseudocódigo de la Sección ?. Los módulos principales son:

- `config.py`: centraliza todos los parámetros globales del experimento, como los intervalos de los parámetros, el número de puntos de la cuadrícula, las opciones del solucionador numérico (tolerancias, número de intentos) y los hiperparámetros del modelo de regresión simbólica (tamaño de la población, número de generaciones, coeficiente de parsimonia, etc.).
- `equation_parser.py`: convierte las ecuaciones del sistema, escritas como cadenas de texto, en expresiones simbólicas de `SymPy` [?] que pueden ser evaluadas numéricamente. Para sistemas de varias ecuaciones, este módulo procesa la lista completa y devuelve tanto las expresiones como un diccionario con todos los símbolos involucrados.
- `grid_generator.py`: construye el producto cartesiano de los conjuntos de valores de los parámetros (Sección ??), generando la cuadrícula $\mathcal{G}$ que se utiliza como entrada en la etapa de resolución numérica.
- `solver.py`: para cada tupla de la cuadrícula, resuelve numéricamente el sistema con `SciPy` [2] empleando múltiples puntos iniciales aleatorios. Aplica el filtro de duplicados basado en la distancia euclidiana (Sección ??) y devuelve la lista de soluciones únicas encontradas para cada valor de los parámetros.
- `root_grouping.py`: reorganiza los datos generados por el solucionador para que puedan ser utilizados por los módulos de regresión simbólica. Produce tanto el conjunto combinado de pares $(\theta, \mathbf{x})$ como, opcionalmente,

la separación por soluciones de acuerdo con la estrategia descrita en la Sección ??.

- **symbolic_regression.py**: contiene la lógica de entrenamiento de PySR [1] y el algoritmo iterativo que, partiendo del conjunto de datos completo, descubre una a una las expresiones para x_1 (Sección ??). Tras cada entrenamiento, se examinan todas las ecuaciones generadas por el modelo y se selecciona aquella que cubre la mayor cantidad de puntos, descartando el resto antes de la siguiente iteración.
- **regression_adapter.py**: implementa la estrategia multidimensional: extiende las expresiones obtenidas para x_1 a las coordenadas restantes (Sección ??) y valida cada solución completa calculando el error medio sobre el sistema original (Sección ??).
- **main.py**: integra los módulos anteriores y ejecuta el pipeline completo, desde la lectura de la configuración hasta la escritura de los resultados.

Además de los módulos desarrollados, el sistema depende de las bibliotecas externas **SymPy** para la manipulación simbólica, **SciPy** para la resolución numérica y **PySR** para la regresión simbólica.

0.2. Entradas, parámetros y salidas del sistema

Para cerrar la descripción del método, se resumen a continuación los datos que requiere el sistema y los resultados que produce.

Entradas del sistema:

- Lista de ecuaciones que definen el sistema $\mathbf{F}(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{0}$.
- Nombres de las variables $\mathbf{x}$ y de los parámetros $\boldsymbol{\theta}$.
- Intervalos $[\theta_i^{\min}, \theta_i^{\max}]$ para cada parámetro, y número de puntos de la cuadrícula.

Parámetros principales:

- Número de puntos iniciales aleatorios y tolerancias para la resolución numérica ($\varepsilon_{\text{res}}, \tau$).
- Función de pérdida seleccionada y sus parámetros (ε, k) (Sección ??).
- Hiperparámetros de PySR: tamaño de población, número de generaciones, coeficiente de parsimonia, tamaño máximo de árbol, etc. (Sección ??).

Salidas del sistema:

- Lista de soluciones encontradas. Cada solución consiste en un conjunto de expresiones simbólicas $\{g_1(\boldsymbol{\theta}), \dots, g_n(\boldsymbol{\theta})\}$ que describen las variables en función de los parámetros.
- Para cada solución se indica el subconjunto de parámetros sobre el cual fue validada y el número de puntos que cubre.

Referencias

- [1] Miles Cranmer. PySR: Fast & parallelized symbolic regression in Python/Julia. *Journal of Open Source Software*, 8(89):5308, 2023.
- [2] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272, 2020.